



Hardware Acceptance Test Version 1.5.0

Checklist to run on at least one real charger before the pilot: connection security, metering and billing accuracy, payments, interruptions, emergency control, compliance states and sign-off.

Build	v1.5.0 (git tag) · plugsure-v1.5.0.zip
Source	docs/ACCEPTANCE-v1.4.md in the package
Charger	OCPP 1.6J, security profile 2 (TLS + AuthorizationKey)
Date	2 October 2026
Audience	Field engineers, the operations lead and the witness signing off the pilot

Hardware acceptance checklist

Run this on at least one real charger before the pilot. It supersedes `ACCEPTANCE-v1.3.md`, which demonstrates the console against the simulator. This checklist covers what only real hardware can show:

- meter accuracy;
- power loss;
- offline transactions;
- connector release time;
- the behaviour v1.4.0 changed.

Tick each box. Record a UTC time, the session or transaction ID and the evidence (a photo, or a console screenshot) against every **Pass**. One failed **Pass** stops the pilot until the failure is explained.

Record sheet

Field	Value
Date / site	
Charger vendor, model, serial	
Firmware version	
Charge point identity	
Connectors (type, kW)	
Reference meter (make, serial, calibration date)	
Test vehicle(s)	
Witnesses	
PlugSure build (git tag, e.g. <code>v1.4.4</code>)	

Equipment:

- a calibrated reference energy meter on the test connector;
- two RFID cards: one registered, one not;
- a phone with the driver app;
- a way to cut the charger's mains supply and its network (WAN cable or SIM) separately;
- a stopwatch.

0. Before you go on site

- ☐ The server runs the published release under test (`v1.4.4` or later; record the tag above), behind Caddy with a Let's Encrypt certificate (`deploy/Caddyfile` re-deployed).

- ☐ The gateway environment has:
 - `NODE_ENV=production`;
 - `OCPP_MIN_SECURITY_PROFILE=2`;
 - `OCPP_AUTO_ADOPT=false`;
 - a 64-hex `SECRETS_KEY`. The gateway refuses to start otherwise.
- ☐ The API and gateway connect as `plugsure_app`. **Pass:** the startup log says *"row-level security is in force for application queries"*.
- ☐ **Health.** On the server, `curl http://127.0.0.1:9220/healthz` (gateway) and `curl http://127.0.0.1:9200/healthz` (API) both return `"ok":true`. From outside, `curl -i https://<ocpp-host>/ocpp/<identity>` returns 426 (the gateway, reached through Caddy, expects a WebSocket upgrade). The supplied Caddyfile answers 404 for any other path on the OCPP host, including `/healthz`.
- ☐ The charger identity is registered in the console. Send the vendor the identity and URL with `deploy/AUTEL-HANDOUT.md`, and send the AuthorizationKey by a separate channel.
- ☐ Take a database backup (`tools/backup/pg-backup.sh`) and note its file name. The test leaves real sessions and invoices behind.
- ☐ Assign a known tariff to the site and write down the expected price. Use one with an energy rate and an idle fee, and if possible a time window that the test session will cross.

1. Connection and security

- ☐ **Wrong key.** Configure the charger with a deliberately wrong AuthorizationKey. **Pass:** the connection is refused with 401, and the console's connection log shows the attempt with its source IP. Restore the right key.
- ☐ **Plain ws://.** If the charger allows it, point it at `ws://`. **Pass:** refused (403, or no connection at all). No session can start over an unencrypted link.
- ☐ **Correct settings:** `wss://.../ocpp/<identity>`, OCPP 1.6J, Profile 2. **Pass:**
 - The WebSocket opens.
 - `BootNotification` is answered `Pending` until an operator activates the charger. Auto-adoption is off.
 - After activation the charger is `Accepted`.
 - The console shows the vendor, model and firmware the charger reported.
- ☐ **Clock.** **Pass:** the charger's clock is within 5 s of UTC after boot. Compare the `Heartbeat` reply time with the charger's display.
- ☐ **Heartbeat.** **Pass:** the charger shows Online, and the heartbeat interval matches the `HeartbeatInterval` the server set (300 s).
- ☐ **Unsolicited certificate request.** If the charger sends `SignCertificate` without being asked, **Pass:** it is refused. Certificates are signed only on an operator's request.

2. Authorisation

- ☐ **Registered card.** **Pass:** Authorize is `Accepted` and charging starts.
- ☐ **Unregistered card.** **Pass:** Authorize is `Invalid`, nothing charges, and no session is billed.

- ☐ **Blocked card.** Block the registered card in the console and push the local list. **Pass:** the charger refuses the card both online and with the network disconnected.
 - ☐ **Remote start/stop from the console.** **Pass:** the charger starts within 10 s and stops within 10 s. Both commands appear in the audit log with the operator's name.
-

3. Metering and billing accuracy

This section decides whether the bill is right, so take your time.

- ☐ Charge for at least 30 minutes and at least 10 kWh. Read the reference meter, the charger display and the console at the start and at the stop.
 - ☐ **Energy.** **Pass:** the console's session energy equals the charger's `meterStop - meterStart` exactly. It is within the meter's accuracy class ($\leq 1\%$ for class 1, $\leq 0.5\%$ for class 0.5S) of the reference meter.
 - ☐ **Final reading.** v1.4.0 bills `meterStop`, not the last periodic sample. **Pass:** the billed energy includes the energy after the last `MeterValues` sample. To make that visible, stop the session a few seconds after a sample.
 - ☐ **No lifetime-register billing.** **Pass:** the billed kWh is the session's energy, not the charger's lifetime register.
 - ☐ **Price.** Work out the expected price by hand from the tariff and compare. **Pass:** the receipt matches to the rupiah, including:
 - the time-window price, applied once, if the session crossed a window;
 - the PBJT-TL, DPP and PPN lines;
 - the rounding line.
 - ☐ **Idle fee.** Leave the car plugged in after charging stops, for longer than the grace period. **Pass:** the idle minutes appear on the receipt at the tariff's idle rate, and the time charge excludes them.
 - ☐ **Receipt.** **Pass:** the session and its receipt appear in the console and in the driver app with the same amount.
-

4. Payments on the charger

- ☐ **Prepaid QR, right connector.** Pay for a session with the QR on connector 1, then plug into connector 1. **Pass:** charging starts, and the session is linked to the payment.
 - ☐ **Prepaid QR, wrong connector.** Pay for connector 1, then try to use the token on connector 2. **Pass:** refused. Nothing charges against that payment.
 - ☐ **Driver app.** Start and stop a session from the app (post-pay or saved card). **Pass:** charged exactly once. Pressing Stop twice, or a network retry, does not charge twice.
 - ☐ **Refund.** Refund the prepaid session from the console. **Pass:** the refund reaches the acquirer once, and a second refund of the same amount is refused.
-

5. Interruptions

Each scenario uses its own session.

- ☐ **Network loss during a session.** Disconnect the charger's WAN for 10 minutes mid-session, reconnect, then stop. **Pass:**
 - Charging carries on locally.
 - The queued `MeterValues` and `StopTransaction` arrive on reconnect, in order.
 - The session closes with the correct energy, and the console shows no duplicate session.
 - ☐ **Offline start, valid card.** With the WAN down, start a session with a card on the local list, then stop it and reconnect. **Pass:** the offline transaction uploads, is recorded once and is billed normally.
 - ☐ **Offline start, refused card.** With the WAN down, start with a card the server refuses but the charger accepts locally (block it on the server only), then reconnect. **Pass:**
 - The session is recorded, not lost, and flagged `needs_review`.
 - It is billed to no one until an operator decides.
 - ☐ **Mains power loss.** Cut the charger's mains power mid-session for 2 minutes, then restore it. **Pass:**
 - The charger reboots and sends `BootNotification`.
 - The interrupted session closes, using the charger's stop data or the last register.
 - The energy is not double-counted when the charger resumes or starts a new transaction.
 - ☐ **Server restart.** Run `systemctl restart plugsure-gateway` mid-session. **Pass:** the charger reconnects within its backoff, and the session continues and closes correctly.
-

6. Emergency and control

- ☐ **Unlock cable, 3 s limit.** During a session, send Unlock from the console and time it from the confirm click to the physical release. **Pass:** ≤ 3 s on three attempts. Record each time.
- ☐ **Power limit.** Set the site budget below the charger's maximum. **Pass:**
 - The charger's measured output falls to the limit within 60 s, checked on the reference meter or vehicle.
 - Values above the PLN contract are clamped in the console.
- ☐ **Station ceiling.** **Pass:** the ceiling (`ChargePointMaxProfile`) cannot be cleared through the raw command route.
- ☐ **Availability.** Set a connector Inoperative, then Operative. **Pass:** the charger refuses to start on it, then accepts again.

7. Compliance states

- ☐ **Suspend and resume.** In the console, open the charger and choose **Suspend...** with a reason. **Pass:**
 - It stays connected and is shown as suspended.
 - A registered card and a remote start are both refused.
 - In the driver app it stays on the map, shown as temporarily out of service; a payment, a reservation and a queue place for it are all refused.
 - After a power cycle it boots `Pending`.
 - **Resume** needs nothing on the charger. If it is connected, the console shows it online at once. If it was power-cycled while suspended, it is `Accepted` at its next `BootNotification`: asked for at once, at the latest within its 5-minute `Pending` interval.
 - Both actions appear in the audit log, the suspension with its reason.
- ☐ **Tera ulang lapsed.** Set one connector's tera expiry date in the past. **Pass:** commercial sessions on that connector are blocked, and a critical alert is raised. Restore the date.
- ☐ **Decommission and reinstate.** Do this last, after section 8, because decommissioning erases the charger's key. With no session running:
 1. Decommission the charger (`POST /v1/charge-points/<identity>/decommission`), then power-cycle it (a connection already open is not dropped by decommissioning). **Pass:** it is refused at connect, and while still connected it cannot start a session.
 2. Reinstate it (`POST /v1/charge-points/<identity>/reinstate`). **Pass:** it returns to awaiting adoption. It needs a new `AuthorizationKey` and Profile 2 again before it can connect, and then boots `Pending` until it is activated.

8. Maintenance features

- ☐ **Configuration.** Read the configuration keys, change `MeterValueSampleInterval` to 60, and read it back. **Pass:** the change takes effect, and `AuthorizationKey` cannot be changed through the raw command route.
- ☐ **Diagnostics.** Request logs with the built-in receiver. **Pass:** the file uploads and opens in the viewer.
- ☐ **Firmware.** Only with the vendor's approval and a known-good image over `https`. Run a single-charger firmware campaign. **Pass:**
 - The tracker reaches *Verified*.
 - The charger reconnects with the new version.
 - A `http://` or internal URL is refused.

9. After the test

- ☐ **Audit trail.** The audit log contains every operator action taken above. **Pass:** `GET /v1/audit` returns `"chain": {"ok": true, ...}` with an empty `problems` list.
- ☐ **Review queue.** Settle every session flagged `needs_review` (approve or void it) and note what you did.

- ☐ **OCPP frames.** Export the frame log for the test window (GET /v1/charge-points/<identity>/frames.ndjson) and keep it with this sheet.
- ☐ **Cleanup.** Void the test invoices if needed, restore the tariff and site settings, and re-enable anything you blocked.

Sign-off

Section	Result (pass / fail / n.a.)	Notes
0. Before you go on site		
1. Connection and security		
2. Authorisation		
3. Metering and billing		
4. Payments		
5. Interruptions		
6. Emergency and control		
7. Compliance states		
8. Maintenance		
9. After the test		

Accepted for pilot by		Date	
Signature			
Witnessed by		Date	
Signature			